

Universidad Distrital Francisco José de Caldas

Systems Engineering Program
Faculty of Engineering

Technical Report

Ecosystem Survival

Cristian Esteban Castañeda Vargas
Julian Carvajal Garnica
Juan Pablo Angulo Guerrero

Group: 8

Supervisor: Carlos A. Sierra Virgüez

Bogotá, Colombia
March 27, 2026

Declaration

We, Cristian Esteban Castañeda Vargas, Julian Carvajal Garnica, and Juan Pablo Angulo Guerrero, students of the Systems Engineering Program, Faculty of Engineering at Universidad Distrital Francisco José de Caldas, declare that this report is entirely our own work.

All figures, tables, code fragments, and illustrations included in this document are original, except where properly cited and referenced. We understand that failure to acknowledge sources constitutes plagiarism, which is considered academic misconduct.

We give consent for this report to be shared with future students as a reference.

We also give consent for this work to be made available for educational and research purposes.

Cristian Esteban Castañeda Vargas
Julian Carvajal Garnica
Juan Pablo Angulo Guerrero

March 27, 2026

Contents

List of Figures	iii
List of Tables	iv
1 Introduction	2
1.1 Background	2
1.2 Problem statement	3
1.3 Aims and objectives	3
1.4 Solution approach	3
1.5 Scope	3
1.6 Assumptions	3
1.7 Limitations	4
1.8 Summary of contributions and achievements	4
1.9 Organization of the report	4
2 Literature Review	5
2.1 Software Architecture for Digital Game Mechanics	5
2.2 Critique of the review	6
2.3 Summary	6
3 Methodology	7
3.1 General Design Approach	7
3.2 Problem Structuring and Decomposition	7
3.3 Algorithm Design Methodology	8
3.3.1 Greedy Algorithm Design	8
3.3.2 Backtracking Algorithm Design	9
3.4 Data Structure Design	10
3.4.1 Linked List for Path Representation	10
3.4.2 AVL Tree for Data Management	11
3.5 JSON Design (File I/O Contract)	12
3.5.1 Input File	12
3.5.2 State File	13
3.6 Interface Design (Wireframe)	13
3.7 Work Distribution	14
3.8 Summary	14
References	15

List of Figures

3.1	System Architecture Diagram	7
3.2	Linked List Diagram	11
3.3	AVL diagram	12
3.4	Ui Wireframe Diagram	13

List of Tables

Abstract

Ecosystem Survival is a grid-based strategy game designed to simulate the management and preservation of species within a dynamic ecosystem. The main objective of the game is to prevent species extinction by making decisions related to feeding and migration, while dealing with limited resources and environmental constraints.

At this stage of the project, the focus is on the conceptual and algorithmic design of the system. Two main algorithmic approaches are proposed: a greedy algorithm to recommend which species should be prioritized based on risk levels, and a backtracking algorithm to determine valid migration paths toward safe zones while avoiding obstacles.

Additionally, the project defines the core data structures required for implementation, including linked lists for path representation and an AVL tree for efficient species management. A JSON-based file contract is also established to handle input actions and represent the game state, ensuring clear communication between components.

This report presents a preliminary design of the system, including algorithm definitions, data structure selection, and interface planning. These elements serve as the foundation for the future implementation of the game in subsequent development stages.

Keywords: greedy algorithm, backtracking, data structures, AVL tree, linked list, game design, ecosystem simulation

GitHub repository

<https://github.com/JulianCarvajalG/ecosystem-survival>

Chapter 1

Introduction

In recent years, computational models and algorithmic approaches have become essential tools for analyzing and solving complex problems in different domains. One particularly relevant area is the management of ecosystems, where multiple variables such as resource availability, environmental risks, and species conditions must be considered when making decisions. These scenarios naturally involve optimization and search problems, making them suitable for algorithmic modeling.

Within this context, the project *Ecosystem Survival* is proposed as a grid-based strategy game that simulates decision-making in a simplified ecosystem environment. The objective of the game is to prevent species extinction by managing limited resources and guiding species toward safe zones. This involves determining which species should be prioritized and how they can safely navigate through a grid containing obstacles and hazards.

The project is conceived as an academic exercise focused on the design and analysis of algorithms. In particular, it emphasizes the application of two fundamental techniques: greedy algorithms and backtracking. The greedy approach is used to recommend which species should be attended first based on a calculated risk value, while the backtracking algorithm is designed to explore possible paths within the grid and determine valid migration routes.

At the current stage of development, the focus is on defining the structure and behavior of the system rather than implementing a complete functional product. This includes the formal definition of algorithms, the selection of appropriate data structures, and the design of a clear interaction model between system components. Additionally, a JSON-based structure is proposed to represent both user actions and the state of the game, ensuring consistency and scalability for future implementation.

The design process has been carried out in a structured manner, beginning with the definition of the game rules and constraints, followed by the formulation of algorithmic solutions and data representations. This approach ensures that all components of the system are well-defined before moving to the implementation phase. The report documents these initial decisions, providing a solid foundation for the subsequent stages of development.

1.1 Background

Ecosystem Survival is designed as a simulation-based game that models the interaction between species and their environment within a constrained grid. The project is motivated by the need to apply algorithmic thinking to problems involving prioritization and pathfinding, which are common in both computer science and real-world scenarios.

Unlike purely entertainment-focused games, this project emphasizes the integration of computational techniques into gameplay mechanics. The use of greedy algorithms allows the system to make efficient decisions based on local optimization criteria, while backtracking enables the exploration of possible solutions in search of valid paths under constraints.

The project also serves as a preparatory step toward implementation, where all major components are defined conceptually before being translated into code.

1.2 Problem statement

In environments where multiple agents must be managed simultaneously, decision-making becomes a complex task that involves evaluating priorities and constraints. In the context of an ecosystem simulation, this translates into deciding which species should be prioritized based on their condition and determining safe paths for their movement within a grid that contains obstacles.

The main problem addressed in this project is how to model these decisions using algorithmic approaches. Specifically, the challenge lies in defining a method to quantify the risk level of each species and selecting the most critical one, as well as designing a strategy to find valid migration paths that avoid hazards and lead to safe zones.

1.3 Aims and objectives

Aim: To design a grid-based ecosystem simulation that applies algorithmic techniques to support decision-making processes.

Objectives:

- Design a greedy algorithm to evaluate and prioritize species based on risk.
- Develop a backtracking algorithm to compute valid migration paths.
- Define data structures suitable for representing paths and species.
- Establish a JSON-based structure for input and state representation.
- Create an initial conceptual design of the user interface.

1.4 Solution approach

The proposed solution is based on a modular design that separates the main components of the system: decision-making, pathfinding, data representation, and user interaction.

The greedy algorithm evaluates each species using a predefined risk function and selects the one with the highest priority. In parallel, the backtracking algorithm explores different paths within the grid to determine a valid route toward a safe zone. These two components complement each other by providing both strategic guidance and operational functionality.

Additionally, the system design includes the definition of data structures that support efficient storage and manipulation of information. A linked list is used to represent migration paths, while an AVL tree is selected for managing species data efficiently.

1.5 Scope

The scope of this stage of the project is limited to the conceptual design of the system. It includes the definition of algorithms, data structures, and interaction models, as well as the design of the file structure used for communication between components.

The implementation of a fully functional game, including graphical interface and real-time interaction, is outside the scope of this milestone and will be addressed in future stages.

1.6 Assumptions

The following assumptions are made for the design of the system:

- The grid size is fixed and known in advance.

- Species follow predefined rules of behavior.
- The environment remains static during each decision step.
- Input data is correctly formatted and valid.

1.7 Limitations

Despite providing a clear conceptual design, this stage of the project has several limitations:

- The system has not yet been fully implemented.
- The model simplifies real-world ecosystem dynamics.
- Performance and scalability have not been evaluated.

1.8 Summary of contributions and achievements

This work presents the initial design of a grid-based ecosystem simulation, including the definition of core algorithms, data structures, and system components. These elements establish a structured foundation for future implementation and further development.

1.9 Organization of the report

This report is organized as follows:

- Chapter 2: Literature review related to algorithmic problem solving and simulation systems.
- Chapter 3: Methodology, including design decisions and system structure.
- Chapter 4: Results of the design phase.
- Chapter 5: Discussion of the proposed approach.
- Chapter 6: Conclusions and future work.

Chapter 2

Literature Review

2.1 Software Architecture for Digital Game Mechanics

The development of digital simulations and games has transitioned from simple, monolithic scripts to complex systems that require robust architectural patterns. Game mechanics—defined as the rules that simulate a virtual world—are what make a simulation unique, but this uniqueness often reduces the opportunity for software reuse. However, as noted by Mizutani, Daros, and Kon [1], while a specific implementation might be unique, a high-level software architecture for those mechanics can and should be reused to ensure system maintainability and scalability.

This review aims to contextualize the development of the *Ecosystem Survival* project within the contemporary landscape of game architecture. To achieve this, it analyzes the fundamental trade-offs between design requirements and architectural practices, such as decoupling logic from presentation. Furthermore, it examines how structured data management and formal design patterns underpin the modularity of the current project, which separates a high-performance C++ engine from a Python-based graphical user interface (GUI).

The scope of modern game development is broad, ranging from simple narrative experiences to complex "Economy" and "Physics" based simulations. For *Ecosystem Survival*, the primary focus is on the "Economy" of the ecosystem—the management of limited resources among competing species. This requires an architecture that can handle "Complex Usage" without suffering from performance degradation. Mizutani et al. [1] identified that researchers frequently favor decoupling and the use of design patterns to manage this complexity. Specifically, by isolating the simulation rules (the "Mechanics") from the visual output, developers can optimize the core logic independently of the interface's rendering constraints.

According to the systematic review of 36 major studies in the field [1], the most effective architectural practices for mechanics-heavy systems include:

1. **Decoupling:** Separating the simulation state from the visualization to allow for independent updates and testing.
2. **Design Patterns:** Implementing established solutions (such as State or Strategy patterns) to manage entity behavior.
3. **Component Reuse:** Creating modular code that can be applied to different types of species or environmental rules.
4. **Formal Modeling:** Using technical structures to define how data flows between the logic engine and the user.

Ecosystem Survival, although an academic-scale project, seeks to avoid the common pitfall of "hardcoding" mechanics—a restriction identified by Mizutani as a major barrier to scalability [1]. Many simulations fail to scale because their rules are tightly bound to the game loop. By implementing a C++ core, this project ensures that the heavy lifting—such as the **AVL Tree** for population

management and **Backtracking** for pathfinding—operates within a high-performance environment. This technical foundation allows the system to handle an increasing number of entities without the "Does not scale well" restriction often found in poorly architected games.

Furthermore, the communication between the Python interface and the C++ engine via JSON files serves as a "Data Contract." This ensures that the simulation remains a "State Machine," where the logic engine calculates the next turn's state based on input, and the GUI simply renders that state. This decoupled approach is a direct application of the findings in [1], which suggest that modularity is the key to managing complex digital mechanics.

2.2 Critique of the review

The review of state-of-the-art architectures identified several critical reflections for the field of game development. On one hand, the shift toward decoupled architectures has significantly improved the "Testability" and "Maintenability" of simulation software. High-level patterns allow researchers to experiment with different survival rules without needing to rebuild the entire visual system [1].

However, the review also highlights significant "Design Restrictions." Many studies report that as mechanics become more realistic or complex, the architecture often "Encourages Hardcoding" to meet performance targets, which ultimately limits the system's flexibility. Despite the benefits of decoupling, the overhead of communication between layers can be a barrier if not managed correctly.

In this context, *Ecosystem Survival* addresses these challenges by using a hybrid approach. By leveraging C++ for the logic (speed) and Python for the GUI (ease of use), it bypasses the performance limitations of high-level languages while maintaining a clean architectural separation. This project demonstrates that applying systematic software architecture principles is essential for creating a reliable and scalable ecological simulation.

2.3 Summary

This chapter presented a review of the architectural requirements and practices for digital game mechanics. By analyzing the findings of Mizutani et al. (2021), the importance of decoupling, modularity, and the avoidance of hardcoded rules was established. *Ecosystem Survival* is proposed as a solution that adheres to these high-level architectural standards, utilizing a decoupled Python-C++ engine to provide a stable, efficient, and maintainable simulation environment.

Chapter 3

Methodology

3.1 General Design Approach

The development of the project follows a structured, design-oriented methodology aligned with the Milestone 1 checkpoint requirements. At this stage, the objective is not to produce executable code, but to define a complete and coherent technical foundation that will guide the implementation phase.

The methodology is based on progressively transforming the conceptual idea of the game into formal representations. This process began with the definition of rules and constraints, followed by the identification of algorithmic problems, and finally the formalization of those problems through pseudocode, data structures, and communication protocols.

An iterative approach was adopted, where ideas were refined through continuous discussion, validation with example scenarios, and consistency checks between components. This ensured that every design decision is justified and aligned with the overall system behavior.

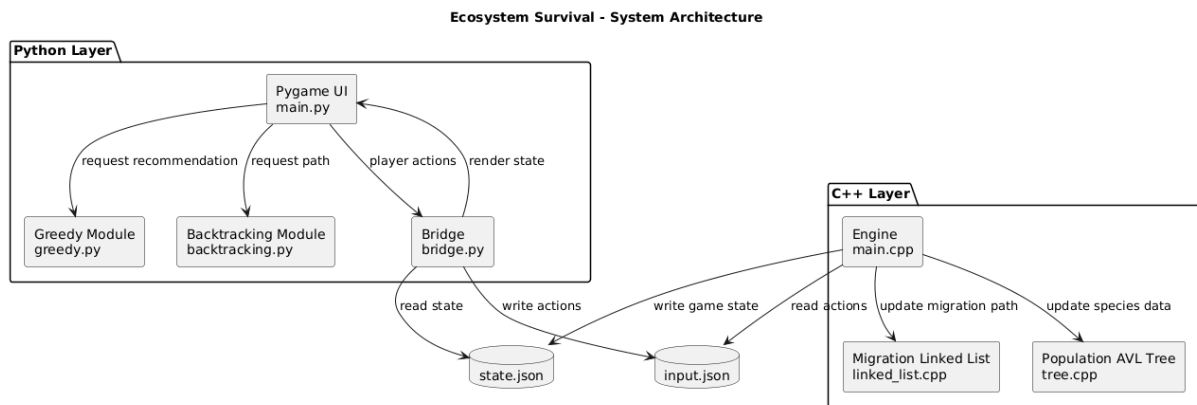


Figure 3.1: System Architecture Diagram

3.2 Problem Structuring and Decomposition

Before designing any technical component, the team first translated the game mechanics into a formal problem structure. This included defining:

- the board representation and its constraints,
- the entities involved and their attributes,
- the movement and interaction rules,
- the win and loss conditions.

Once these elements were established, the problem was decomposed into smaller, manageable subproblems. Two key computational challenges were identified:

- decision-making under priority conditions (greedy approach),
- constrained pathfinding within a grid (backtracking approach).

This decomposition allowed the team to clearly determine where algorithms were required and how they interact with the rest of the system.

3.3 Algorithm Design Methodology

The design of the algorithms followed a three-stage process: conceptual definition, logical decomposition, and formal pseudocode construction. This ensured that the algorithms are both understandable and directly translatable into code in later stages.

3.3.1 Greedy Algorithm Design

The greedy algorithm was designed as a recommendation system to assist decision-making within the game. The main challenge was defining a quantitative measure of priority.

Through analysis, the team identified two critical factors:

- **Urgency:** represented by the hunger level,
- **Vulnerability:** represented by the population size.

These factors were combined into a single evaluation metric, allowing the algorithm to compare entities objectively.

The logic of the algorithm consists of scanning all available entities, computing a score for each one, and selecting the entity with the highest score.

```

1 function GreedyRecommend(species_list):
2
3     best_species <- null
4     highest_value <- -1
5
6     for each species in species_list:
7
8         if species.status != "extinct" and species.status != "safe":
9
10            value <- species.hunger + (10 - species.population)
11
12            if value > highest_value:
13                highest_value <- value
14                best_species <- species
15
16     return best_species

```

Listing 3.1: Greedy algorithm pseudocode

This algorithm was intentionally designed to be efficient and deterministic. By performing a single pass over the data, it achieves linear time complexity, making it suitable for real-time recommendations during gameplay.

3.3.2 Backtracking Algorithm Design

The backtracking algorithm was designed to solve a constrained pathfinding problem. The objective is to find a valid path from a starting position to a target cell while respecting multiple restrictions.

The design process began by defining what constitutes a valid move:

- the position must be inside the board,
- it must not have been visited before,
- it must not be a blocked cell,
- it must not contain another entity.

Once these conditions were established, the solution was modeled as a recursive exploration of possible paths. The algorithm attempts all valid directions, builds a path incrementally, and discards invalid paths through backtracking.

```

1 function FindPath(board, row, col, visited, path):
2
3     if board[row][col] == "Z":
4         return true
5
6     visited[row][col] <- true
7
8     for each (dr, dc) in directions:
9
10        new_row <- row + dr
11        new_col <- col + dc
12
13        if IsValid(board, new_row, new_col, visited):
14
15            append (new_row, new_col) to path
16
17            if FindPath(board, new_row, new_col, visited, path):
18                return true
19
20            remove last element from path
21
22    return false

```

Listing 3.2: Backtracking pathfinding pseudocode

```

1 function IsValid(board, row, col, visited):
2
3     if row < 0 or row >= board_size:
4         return false
5
6     if col < 0 or col >= board_size:
7         return false
8
9     if visited[row][col] == true:
10        return false
11
12    if board[row][col] == "X":
13        return false
14
15    if board[row][col] is a species:
16        return false
17
18    return true

```

Listing 3.3: Validation function pseudocode

This recursive structure allows the algorithm to explore all feasible paths systematically. The inclusion of a visited matrix prevents infinite loops, while the backtracking step ensures that incorrect paths are discarded efficiently.

3.4 Data Structure Design

The data structures were selected based on the operational needs of the algorithms and the dynamic nature of the game state.

3.4.1 Linked List for Path Representation

The path generated by the backtracking algorithm is represented using a linked list. This decision was made because paths are sequential and their size is not fixed in advance.

Each node stores a position and a reference to the next node, allowing dynamic growth as the path is constructed.

```
1 struct PathNode {  
2     int row;  
3     int col;  
4     PathNode* next;  
5 };
```

Listing 3.4: Linked list node structure

This structure enables efficient traversal and preserves the order of the path from origin to destination.

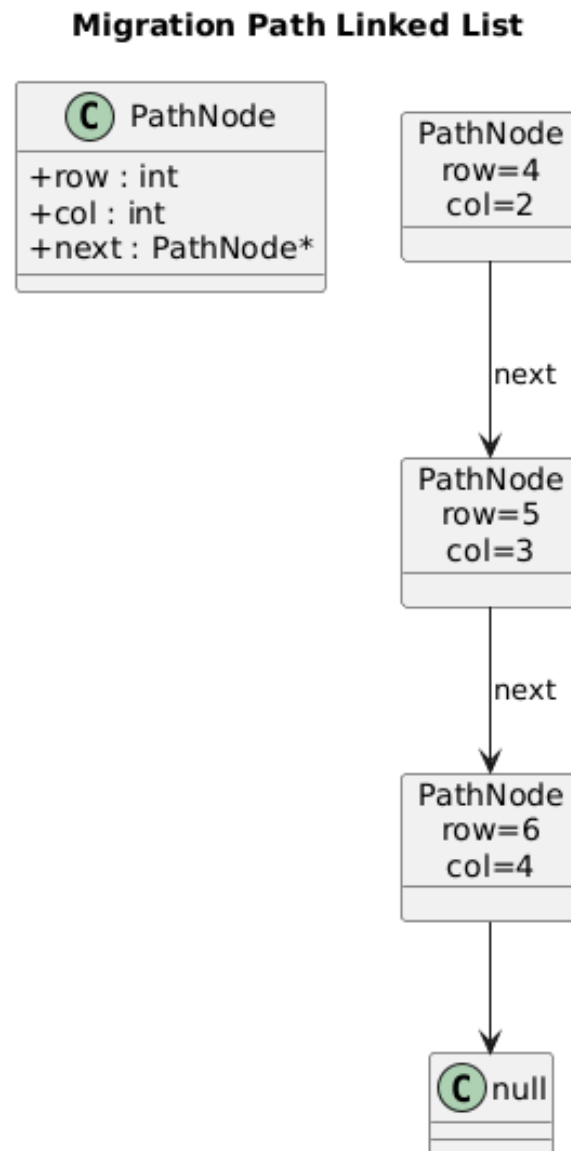


Figure 3.2: Linked List Diagram

3.4.2 AVL Tree for Data Management

To efficiently manage dynamic data, a tree structure was required. The team evaluated the use of a Binary Search Tree (BST) versus an AVL tree.

A BST, while simple, can become unbalanced and degrade to linear performance. In contrast, an AVL tree maintains balance through rotations, ensuring logarithmic time complexity for all operations.

```

1 struct AVLNode {
2     string key;
3     int value;
4     int height;
5     AVLNode* left;
6     AVLNode* right;
7 };
  
```

Listing 3.5: AVL tree node structure

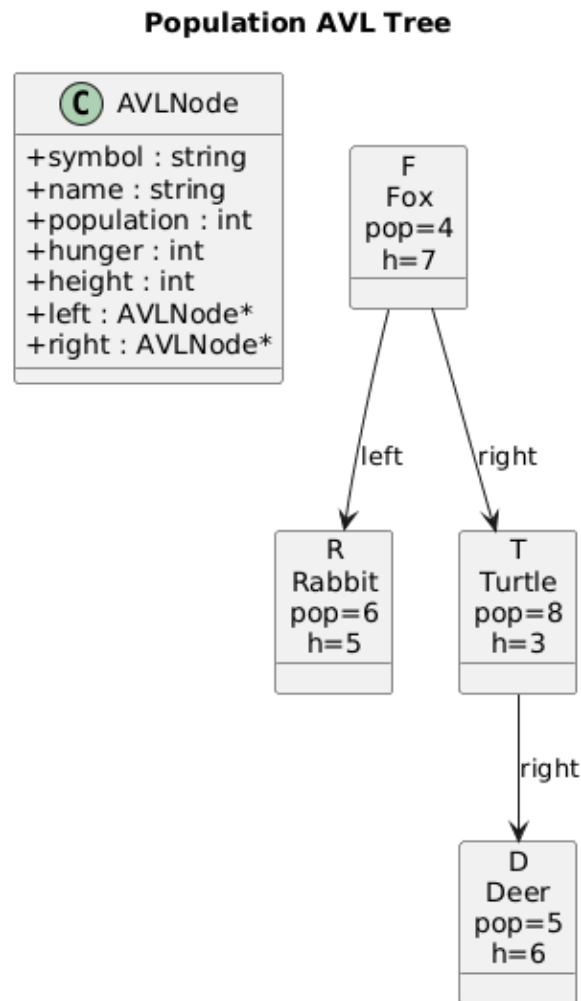


Figure 3.3: AVL diagram

The selection of AVL ensures consistent performance for insertion, deletion, and search operations, which is essential for a dynamic system.

3.5 JSON Design (File I/O Contract)

The system uses a file-based communication model through JSON files. This design ensures clear separation between components and simplifies integration.

The design process involved identifying all the information required for both input actions and system state representation.

3.5.1 Input File

The input file defines player actions in a simple and structured format.

```

1 {
2   "action": "feed",
3   "species": "R"
4 }
```

Listing 3.6: Example input.json

3.5.2 State File

The state file contains all the information required to represent the current state of the game.

```

1 {
2   "grid_size": 8,
3   "turn": 1,
4   "actions_left": 2,
5   "board": [
6     [" ", "R", " ", " ", " ", "X", "Z"],
7     {
8       "symbol": "R",
9       "population": 5,
10      "hunger": 3,
11      "position": [0,1],
12      "status": "active"
13    }
14  ]
15 }
```

Listing 3.7: Example state.json

The structure was designed to ensure that all necessary information is available for both logic processing and graphical rendering.

3.6 Interface Design (Wireframe)

The interface was planned using a wireframe that defines the spatial distribution of elements within the screen.

The design includes:

- a grid representing the board,
- an information panel displaying entity data,
- indicators for turn and available actions,
- visual feedback for algorithm recommendations.

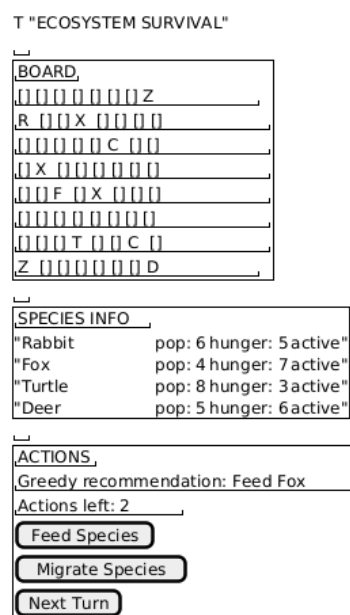


Figure 3.4: Ui Wireframe Diagram

This approach ensures that the interface supports all required functionalities before implementation begins.

3.7 Work Distribution

The team organized responsibilities to ensure efficient progress and clear ownership of tasks.

Each member is assigned a primary role corresponding to the main components of the system, while maintaining a shared understanding of the entire project.

This structure facilitates parallel development and reduces integration issues in later stages.

3.8 Summary

The methodology combines conceptual reasoning with formal technical design. By including both detailed explanations and pseudocode representations, the project establishes a solid and consistent foundation for future implementation.

This approach ensures that all components are well-defined, interconnected, and ready to be translated into a functional system in subsequent stages.

References

- [1] W. K. Mizutani, V. K. Daros, and F. Kon, "Software Architecture for Digital Game Mechanics: A Systematic Literature Review," *Entertainment Computing*, vol. 38, p. 100421, 2021. Available: <https://doi.org/10.1016/j.entcom.2021.100421>
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.